

Thermodynamic State Vector Interface and Monad State Transitions in Planetary Ecosystem Simulations

Web of Life Scientific Communications & Academic Outreach
<https://github.com/pascalranoroarijaona/WebOfLife>

Sprint 008

Abstract

Ecosystem simulation models frequently struggle to maintain physical rigor when coupling metabolic kinetics, nutrient cycling, and radiative fluxes. In Sprint 008, we establish the formal thermodynamic state vector interface (`src/thermodynamics/types.ts`) for the Web of Life framework. By enforcing strict mathematical contracts for internal entropy generation ($\dot{S}_{\text{gen}} \geq 0$), exergy destruction rate ($\dot{I} = T_0 \dot{S}_{\text{gen}}$), and boundary heat flux vectors, we guarantee absolute compliance with the First and Second Laws of Thermodynamics. Furthermore, we introduce a monad-like state wrapper (`ThermodynamicMonad`) that threads mass inventories and energy states while intercepting non-physical state transitions at runtime. This paper outlines the theoretical framework, interface specifications, and verification protocols governing open, solar-driven biogeochemical systems.

1 Introduction & Theoretical Framework

The Web of Life simulation architecture models planetary ecosystems as open, non-equilibrium thermodynamic systems driven by solar radiation flux ($\dot{Q}_{\text{solar}}$) and bounded by longwave infrared radiative cooling ($\dot{Q}_{\text{IR}}$). To prevent thermodynamic drift and ensure biological plausibility across nutrient cycles (Carbon, Nitrogen, Phosphorus, Water), all simulation components must adhere to rigorous conservation and dissipation laws.

1.1 First Law of Thermodynamics (Energy Conservation)

The total internal energy change within any discrete control volume V is governed by:

$$\frac{dE}{dt} = \sum \dot{Q}_{\text{in}} - \sum \dot{Q}_{\text{out}} + \sum_{\text{in}} \dot{m} \left(h + \frac{1}{2}v^2 + gz \right) - \sum_{\text{out}} \dot{m} \left(h + \frac{1}{2}v^2 + gz \right) \quad (1)$$

In our Earth Pod model, total elemental mass is strictly conserved, while energy enters via solar radiation and exits via deep-space infrared cooling.

1.2 Second Law & Exergy Destruction

The Gouy-Stodola theorem extension governs system entropy balance:

$$\frac{dS}{dt} = \sum_k \frac{\dot{Q}_k}{T_k} + \dot{S}_{\text{gen}} \quad (2)$$

where $\dot{S}_{\text{gen}} \geq 0$ strictly according to the Second Law. The rate of exergy destruction ($\dot{I}$), quantifying thermodynamic irreversibility relative to standard ambient reference temperature $T_0 = 288.15$ K, is defined as:

$$\dot{I} = T_0 \dot{S}_{\text{gen}} \quad (3)$$

2 Core Interface Contracts

The TypeScript interface definitions codify these physical laws into software engineering contracts within `src/thermodynamics/types.ts`:

```
export interface IThermodynamicStateVector {
  readonly timestamp: number;
  readonly T_0: number; // Default: 288.15 K
  readonly internalEnergy: number; // Joules (J)
  readonly entropy: number; // Joules per Kelvin (J/K)
  readonly entropyGenerationRate: number; // Watts per Kelvin (W/K), >=
    0
  readonly exergyDestructionRate: number; // Watts (W), T_0 *
    entropyGenerationRate
  readonly boundaryHeatFlux: BoundaryHeatFluxArray;
  readonly massInventory: Record<string, number>; // kg across
    biogeochemical cycles
}

export interface BoundaryHeatFluxArray {
  solarIn: number; // >= 0
  infraredOut: number; // <= 0
  sensibleLatentFlux: number; // Convective/Evaporative exchange
}

export interface IThermodynamicSystem {
  getStateVector(): IThermodynamicStateVector;
  calculateEntropyGeneration(dt: number): number;
  verifyFirstLaw(previousState: IThermodynamicStateVector, currentState
    : IThermodynamicStateVector, dt: number): boolean;
  verifySecondLaw(): boolean;
}
```

3 Monad State Transitions & Invariant Enforcement

To propagate thermodynamic states safely across biogeochemical transformations, we implement `ThermodynamicMonad<T>`. This functional wrapper intercepts state mutations, verifying that $\dot{S}_{\text{gen}} \geq 0$ and $\dot{I} \equiv T_0 \dot{S}_{\text{gen}}$ before committing transitions:

```
export class ThermodynamicMonad<T> {
  private constructor(
    private readonly value: T,
    private readonly stateVector: IThermodynamicStateVector
  ) {}

  public static unit<T>(value: T, initialVector:
    IThermodynamicStateVector): ThermodynamicMonad<T> {
    return new ThermodynamicMonad(value, initialVector);
  }

  public bind<U>(
    fn: (val: T, vector: IThermodynamicStateVector) => { value: U;
      vector: IThermodynamicStateVector }
  ): ThermodynamicMonad<U> {
    const result = fn(this.value, this.stateVector);

    if (result.vector.entropyGenerationRate < 0) {
```

```

        throw new Error('Second Law Violation: S_gen (${result.vector.
            entropyGenerationRate}) < 0 W/K');
    }

    const expectedExergyDestruction = result.vector.T_0 * result.vector
        .entropyGenerationRate;
    if (Math.abs(result.vector.exergyDestructionRate -
        expectedExergyDestruction) > 1e-6) {
        throw new Error('Exergy Inconsistency: I != T_0 * S_gen');
    }

    return new ThermodynamicMonad(result.value, result.vector);
}

public getStateVector(): IThermodynamicStateVector { return this.
    stateVector; }
public getValue(): T { return this.value; }
}

```

4 Verification Strategy

1. **Unit Testing:** Rigorous validation of $\dot{S}_{\text{gen}} \geq 0$ under extreme metabolic loads.
2. **Exergy Consistency:** Strict assertion of $\dot{I} = T_0 \dot{S}_{\text{gen}}$ within 10^{-6} W tolerance.
3. **Mass Conservation:** Tracking C, N, P, and H₂O inventories within 10^{-9} relative precision.

5 Conclusion

Sprint 008 establishes an immutable mathematical and computational foundation for simulating planetary ecosystems under strict thermodynamic constraints. By combining strict interface typing with monad-based invariant enforcement, the Web of Life framework prevents unphysical simulations and guarantees robust systems ecology modeling.

Repository Reference: <https://github.com/pascalranoroarijaona/WebOfLife>